

Encoding Numerical Data

Module 32 Study Notes • Binning and Binarization

1. Why Encode Numerical Data?

Until now, we focused on converting categorical text into numbers. But occasionally, we need to do the exact opposite: convert continuous numerical data into discrete categories. Why?

Real-World Example: Imagine a Google Play Store dataset with a "Downloads" column. You have apps with 5 downloads and apps with 1,000,000,000 downloads. Plotting this creates a massive, skewed mess. By converting these numbers into discrete categories or "bins" (e.g., *<10k*, *10k-1M*, *>1M*), you drastically simplify the pattern for your machine learning model to understand.

2. Discretization (Binning)

Discretization is the process of transforming continuous variables into discrete ones by creating a set of intervals (bins) that span the range of the variable's values.

A. Equal Width (Uniform) Binning

This splits the data range into N intervals of exactly the same size.

- **Formula:** $\text{Bin Width} = (\text{Max_Value} - \text{Min_Value}) / \text{Number_of_Bins}$
- **Pros:** Excellent for handling extreme outliers (outliers get grouped into the first or last bins).
- **Cons:** It does not change the original spread or shape of the data. If your data is heavily skewed, most of your data points might end up crammed into a single bin.

B. Equal Frequency (Quantile) Binning

This creates bins such that each bin contains roughly the *exact same number of data points* (e.g., 10 bins, where each bin holds 10% of the population).

- **Pros:** This is highly effective. Not only does it handle outliers, but it also forces the data spread to become uniform, completely resolving heavy skewness.
- **Usage:** This is generally the default choice when performing binning.

C. KMeans Binning

This leverages the KMeans clustering algorithm. It looks for natural clusters or groupings in your 1D numerical data.

- **Usage:** Use this exclusively when your data's distribution naturally forms distinct groups or islands (multi-modal distribution).

D. Custom / Domain Binning

Sometimes you ignore mathematical algorithms and use pure business logic. For example, binning Age: *0-18 (Child)*, *18-60 (Adult)*, *60+ (Senior)*. Scikit-Learn doesn't have a direct class for this; you must use Pandas `pd.cut()` to write the custom logic yourself.

3. Implementation: KBinsDiscretizer

Scikit-Learn provides the `KBinsDiscretizer` class to automate the first three unsupervised binning strategies.

```
from sklearn.preprocessing import KBinsDiscretizer
from sklearn.compose import ColumnTransformer

# 1. Initialize Discretizer
# strategy options: 'uniform', 'quantile', 'kmeans'
# encode options: 'ordinal' (returns 1 column 0,1,2), 'onehot' (returns N columns)
kbin = KBinsDiscretizer(n_bins=10, encode='ordinal', strategy='quantile')

# 2. Add to ColumnTransformer
ct = ColumnTransformer(
    transformers=[
        ('bin_age', kbin, ['Age'])
    ],
    remainder='passthrough'
)

# 3. Fit and Transform
X_train_transformed = ct.fit_transform(X_train)
```

4. Binarization

Binarization is a specialized, extreme case of discretization where you create exactly **two bins**: 0 and 1. You define a single numerical **threshold**.

- **Values > Threshold** become **1**.
- **Values <= Threshold** become **0**.

Example: In Image Processing, converting a grayscale image to pure black-and-white. Any pixel value above 127 becomes 1 (white), and everything below becomes 0 (black).

```
from sklearn.preprocessing import Binarizer

# E.g., Family_Size > 0 means traveling with family (1), == 0 means alone (0)
binarizer = Binarizer(threshold=0, copy=False)
X_train['Family_Size'] = binarizer.fit_transform(X_train[['Family_Size']])
```